

Comunicación MQTT sobre TLS

Procedimiento detallado del intercambio de datos entre un *publisher* y un *subscriber*

Proyecto: Encriptación con sistema dinámico de Rössler

August 17, 2026

Abstract

Este documento describe cómo dos dispositivos intercambian información utilizando el protocolo MQTT cuando dicho intercambio se protege con TLS. Se detalla el papel del *broker*, la negociación de la sesión segura, el uso de certificados y la forma en que un mensaje publicado por el emisor llega al receptor. La descripción se apoya en la configuración empleada en el programa `Maestro_TLS_Principal.py`, que actúa como *publisher* de los datos cifrados del experimento. El objetivo es comprender el mecanismo de transporte y su propósito dentro del experimento propuesto.

Contents

1	Introducción	2
2	Elementos del sistema	2
2.1	Publisher, subscriber y broker	2
2.2	Topics utilizados	3
2.3	Puerto y certificados	3
3	Establecimiento de la sesión segura	3
3.1	Secuencia del handshake	3
3.2	Qué garantiza y qué no garantiza TLS	4
4	Configuración TLS en el código	4
5	Transmisión de los datos	5
5.1	Preparación del mensaje	5
5.2	Publicación	6
5.3	Recorrido completo de un mensaje	6
5.4	Cierre de la conexión	7
6	Ejemplo comparativo: con y sin TLS	7
7	Papel de TLS dentro del experimento	8
8	Resumen del procedimiento	8

1 Introducción

El experimento genera una imagen cifrada mediante un esquema caótico basado en el sistema de Rössler y en un mapa logístico. Una vez cifrada, esa información debe viajar desde el dispositivo que la produce (un Raspberry Pi 4 que actúa como **maestro**) hasta el dispositivo que la reconstruye (el **esclavo**). El protocolo elegido para ese transporte es MQTT, y el canal se protege con TLS.

Es importante separar dos capas que resuelven problemas distintos:

- El **cifrado caótico** protege el *contenido* de la imagen. Es la aportación del experimento y es independiente de cómo se transporte el resultado.
- El **cifrado TLS** protege el *transporte*. Impide que un tercero conectado a la misma red pueda leer, alterar o suplantar los mensajes MQTT mientras viajan.

Ambas capas se aplican de forma sucesiva: el programa cifra primero la imagen y después entrega el resultado a MQTT, que a su vez lo entrega a TLS. Este documento se ocupa exclusivamente de la segunda capa.

2 Elementos del sistema

2.1 Publisher, subscriber y broker

MQTT no conecta directamente a los dos dispositivos. Introduce un intermediario llamado **broker**, que es un programa servidor (en este proyecto, Mosquitto, ejecutándose en el mismo Raspberry Pi). El flujo es siempre en dos tramos:

1. El **publisher** se conecta al broker y le entrega un mensaje asociado a un *topic*.
2. El **subscriber**, que previamente se conectó al broker y declaró interés en ese *topic*, recibe una copia del mensaje.

Las consecuencias de este diseño son relevantes para el experimento:

- Publisher y subscriber **nunca se conectan entre sí**. No necesitan conocer la dirección del otro, ni estar activos al mismo tiempo.
- Cada dispositivo establece **su propia sesión TLS** con el broker. Existen, por tanto, dos túneles TLS independientes, no uno solo de extremo a extremo.
- El broker **sí ve el contenido** de los mensajes en claro, porque descifra el tramo TLS entrante y vuelve a cifrar el tramo saliente. En este experimento eso no compromete la imagen, porque lo que se publica ya está cifrado caóticamente antes de entrar a MQTT.

2.2 Topics utilizados

Un *topic* es una etiqueta jerárquica de texto que clasifica los mensajes. En el programa se emplean dos:

Topic	Contenido
chaoskeystream/keys	Parámetros del sistema de Rössler (a , b , c) y del mapa logístico.
chaoskeystream/data	Vector cifrado, trayectorias del maestro, dimensiones de la imagen y parámetros de sincronización.

Separar ambos topics permite que el subscriber trate la información de manera distinta: los parámetros son pequeños y estables, mientras que los datos son voluminosos y cambian en cada ejecución.

2.3 Puerto y certificados

La configuración del programa fija tres elementos que definen el canal seguro:

```
1 BROKER = "raspberrypiJED.local"
2 PORT = 8883
3 CA_CERT_PATH = "/etc/mosquitto/ca_certificates/ca.crt"
```

Listing 1: Configuración del canal en `Maestro_TLS_Principal.py`

El puerto **8883** es el asignado por convención a MQTT sobre TLS; el puerto 1883, usado en las pruebas de referencia sin cifrado, transporta los mismos mensajes en texto plano. El archivo `ca.crt` es el certificado de la **autoridad certificadora** (CA) que firmó el certificado del broker: es la pieza que permite al cliente decidir si el servidor al que se conectó es realmente el que dice ser.

3 Establecimiento de la sesión segura

Antes de que se transmita un solo byte de MQTT, cliente y broker deben construir un canal cifrado. Ese proceso se denomina *handshake* TLS y ocurre una sola vez por conexión.

3.1 Secuencia del handshake

1. **ClientHello.** El cliente abre una conexión TCP al puerto 8883 y anuncia qué versiones de TLS soporta, qué combinaciones de algoritmos (*cipher suites*) puede utilizar y un número aleatorio, $random_C$.
2. **ServerHello y certificado.** El broker responde eligiendo una versión y una *cipher suite* de entre las ofrecidas, envía su propio número aleatorio $random_S$ y adjunta su **certificado digital**. El certificado contiene la clave pública del broker, su identidad (el nombre `raspberrypiJED.local`) y la firma de la CA.
3. **Validación del certificado.** El cliente comprueba, usando el `ca.crt` que tiene almacenado localmente, que la firma del certificado es auténtica, que no ha caducado y que el nombre declarado coincide con el servidor al que se conectó. Si cualquiera de estas comprobaciones falla, la conexión se aborta.

4. **Intercambio de claves.** Cliente y servidor intercambian el material criptográfico necesario (típicamente mediante un acuerdo Diffie-Hellman efímero) para obtener un secreto compartido que nunca viaja explícitamente por la red.
5. **Derivación de claves de sesión.** A partir de ese secreto compartido y de los dos números aleatorios, ambos extremos aplican de forma independiente una función de derivación de claves (KDF) y obtienen el mismo conjunto de claves simétricas. Dos de ellas gobiernan el cifrado del tráfico:
 - K_c : cifra lo que el cliente envía al broker.
 - K_s : cifra lo que el broker envía al cliente.

Se emplean claves distintas por sentido para que el mismo material criptográfico no se reutilice en ambas direcciones.

6. **Confirmación.** Cada extremo envía un mensaje `Finished` ya cifrado, que resume todo el intercambio anterior. Si el otro extremo lo descifra y verifica correctamente, queda demostrado que ambos derivaron las mismas claves y que nadie manipuló los mensajes previos.

A partir de este punto, la conexión TCP se comporta como un **túnel cifrado**: todo lo que se escriba en ella se cifra automáticamente antes de salir y se descifra al llegar.

3.2 Qué garantiza y qué no garantiza TLS

Propiedad	Significado en este experimento
Confidencialidad	Un observador en la red Wi-Fi captura únicamente bytes cifrados; no puede recuperar el vector cifrado ni los parámetros de Rössler.
Integridad	Cualquier modificación de un byte en tránsito invalida la verificación y el registro se descarta.
Autenticación del servidor	El cliente confirma que habla con el broker legítimo y no con un impostor que haya suplantado la dirección.
No cubre	El almacenamiento en el broker, ni la protección del contenido frente al propio broker, ni la seguridad del archivo una vez recibido.

4 Configuración TLS en el código

En el programa, toda la negociación descrita se activa con tres líneas:

```

1 client = mqtt.Client()
2 client.tls_set(ca_certs=CA_CERT_PATH, tls_version=ssl.
  PROTOCOL_TLS_CLIENT)
3 client.tls_insecure_set(False)
4 client.connect(BROKER, PORT, 60)
```

Listing 2: Activación de TLS antes de conectar

El significado de cada instrucción es el siguiente:

- `tls_set(ca_certs=...)` indica a la biblioteca cuál es el certificado de la CA en el que debe confiar. Sin este parámetro el cliente no tendría criterio para aceptar o rechazar el certificado del broker.
- `tls_version=ssl.PROTOCOL_TLS_CLIENT` selecciona el modo cliente y habilita, por defecto, la verificación del certificado y del nombre del servidor.
- `tls_insecure_set(False)` exige explícitamente que el nombre contenido en el certificado coincida con `raspberrypiJED.local`. Con el valor `True` el tráfico seguiría cifrado, pero se aceptaría cualquier servidor: se conservaría la confidencialidad y se perdería la autenticación.
- `connect(BROKER, PORT, 60)` abre la conexión TCP, dispara el *handshake* completo y, solo si este concluye correctamente, envía el paquete MQTT CONNECT. El valor 60 es el *keep-alive* en segundos.

Es importante notar el orden: **TLS se establece antes que MQTT**. El paquete CONNECT de MQTT ya viaja cifrado con K_c , y la respuesta CONNACK del broker viaja cifrada con K_s .

5 Transmisión de los datos

5.1 Preparación del mensaje

MQTT transporta el payload como una secuencia de bytes, sin interpretarlo. Los arreglos de NumPy no son directamente transmisibles, por lo que el programa los convierte primero a listas de Python y después a texto JSON:

```

1 data = {
2     "vector_cifrado": vector_cifrado.tolist(),
3     "x_maestro": x_master.tolist(),
4     "y_maestro": y_maestro.tolist(),
5     "z_maestro": z_master.tolist(),
6     "t_maestro": t_maestro.tolist(),
7     "ancho": ancho,
8     "alto": alto,
9     "nmax": nmax,
10    "tiempo_sinc": TIEMPO_SINC,
11    "KEYSTREAM": KEYSTREAM
12 }
```

Listing 3: Construcción del payload

La llamada `json.dumps(data)` produce una única cadena de texto. Esta representación es legible y portable, pero costosa en tamaño: un número en coma flotante que ocupa 8 bytes en memoria puede requerir cerca de 20 caracteres en JSON. Para una imagen de $n_{\max}$ valores, el payload resultante es del orden de decenas de megabytes, lo cual explica que el tiempo de publicación sea una métrica que el programa registra explícitamente.

5.2 Publicación

```

1 client.publish(TOPIC_KEYS, json.dumps({
2     "ROSSLER_PARAMS": ROSSLER_PARAMS,
3     "LOGISTIC_PARAMS": LOGISTIC_PARAMS
4     })), qos=QOS, retain=True)
5 time.sleep(0.5)
6
7 client.publish(TOPIC_DATA, json.dumps(data), qos=QOS, retain=True
8     )

```

Listing 4: Publicación de parámetros y datos

Dos parámetros condicionan el comportamiento de estas publicaciones.

QoS (calidad de servicio). El programa usa $QOS = 1$, que corresponde a la entrega *al menos una vez*. El publisher envía el mensaje y espera del broker una confirmación (PUBACK); si no la recibe dentro del plazo previsto, reenvía. Los tres niveles disponibles son:

Nivel	Semántica	Comportamiento
0	Como máximo una vez	Se envía sin confirmación; puede perderse.
1	Al menos una vez	Se confirma; puede llegar duplicado si se reenvía.
2	Exactamente una vez	Intercambio de cuatro paquetes; sin pérdidas ni duplicados, con mayor latencia.

El nivel 1 es adecuado aquí porque la pérdida del vector cifrado impediría la reconstrucción de la imagen, mientras que un eventual duplicado simplemente sobrescribe el mismo contenido.

Retain. Con `retain=True`, el broker conserva el último mensaje publicado en cada topic. Un subscriber que se conecte después de la publicación lo recibe de inmediato, sin esperar a la siguiente ejecución del maestro. Esto desacopla temporalmente a los dos dispositivos: el esclavo puede iniciarse minutos más tarde y aun así obtener los datos.

La pausa `time.sleep(0.5)` entre ambas publicaciones da margen a que el envío se complete antes de encolar el siguiente, evitando que el mensaje voluminoso de datos compita con el de parámetros.

5.3 Recorrido completo de un mensaje

Suponiendo que ambos dispositivos ya establecieron su sesión TLS con el broker, la publicación del vector cifrado sigue esta secuencia:

1. El publisher construye un paquete MQTT PUBLISH que contiene el topic `chaoskeystream/d` y el payload JSON.
2. TLS fragmenta ese paquete en registros, cifra cada uno con K_c y añade a cada registro un código de autenticación que protege su integridad.
3. Los registros viajan por la red como segmentos TCP ordinarios. Un observador ve tráfico hacia el puerto 8883 y puede medir su volumen, pero no leer su contenido.

4. El broker descifra los registros con K_c , verifica su integridad y reconstruye el paquete PUBLISH original.
5. El broker consulta su tabla de suscripciones, identifica al subscriber registrado en ese topic y almacena el mensaje como *retained*.
6. El broker construye un nuevo paquete PUBLISH hacia el subscriber y lo cifra con la clave de escritura correspondiente a **esa otra sesión TLS**, que es distinta de K_s .
7. El subscriber descifra, obtiene la cadena JSON, la convierte de vuelta a arreglos con `json.loads` y procede al descifrado caótico.

El punto 4 y el punto 6 son los que evidencian que el broker es un punto de terminación TLS: el mensaje se descifra y se vuelve a cifrar en su interior.

5.4 Cierre de la conexión

```
1 client.disconnect()
```

Listing 5: Desconexión ordenada

La llamada envía un paquete MQTT DISCONNECT y cierra la sesión TLS de forma ordenada. Las claves K_c y K_s se descartan: una conexión posterior repetirá el *handshake* completo y derivará claves nuevas. Los mensajes con `retain=True` permanecen almacenados en el broker aunque el publisher se haya desconectado.

6 Ejemplo comparativo: con y sin TLS

El proyecto incluye ejecuciones equivalentes sobre el puerto 1883 (sin TLS) y sobre el 8883 (con TLS). La diferencia observable desde la red es la siguiente.

Puerto 1883. Una captura de tráfico muestra directamente el nombre del topic y el payload JSON legible:

```
chaoskeystream/keys {"ROSSLER_PARAMS": {"a": 0.2, "b": 0.2, "c":
5.7}, ...}
```

Los parámetros del sistema caótico quedan expuestos, y con ellos la posibilidad de reproducir el keystream y deshacer el cifrado.

Puerto 8883. La misma captura muestra únicamente registros TLS:

```
17 03 03 04 A1 | 3f e2 91 c4 7b 0a ... (bytes cifrados)
```

El primer byte identifica el tipo de registro, los dos siguientes la versión, los dos posteriores la longitud, y el resto es contenido cifrado. Ni el topic ni el payload son recuperables sin las claves de sesión.

Costo. El cifrado no es gratuito: el *handshake* añade varios intercambios de mensajes antes de la primera publicación, y cada registro incorpora unos pocos bytes de sobrecarga. El programa cuantifica este efecto mediante la métrica `tiempo_mqtt_segundos`, que se registra en `tiempos_procesos.csv` junto con los tiempos de difusión y confusión, permitiendo comparar directamente ambas configuraciones.

7 Papel de TLS dentro del experimento

La combinación de ambas capas responde a amenazas distintas:

- Sin cifrado caótico y sin TLS, cualquier observador de la red recupera la imagen íntegra.
- Con cifrado caótico pero sin TLS, la imagen viaja ilegible, pero los parámetros publicados en `chaoskeystream/keys` viajan en claro y permiten reconstruir el keystream. La protección resulta ineficaz.
- Con ambas capas, el observador no obtiene ni el contenido ni los parámetros, y no puede alterar los mensajes sin que la manipulación sea detectada.

TLS resuelve, por tanto, el problema práctico que el esquema caótico no aborda por sí solo: la distribución segura de los parámetros que actúan como clave. Esta separación de responsabilidades permite además evaluar el esquema de cifrado de forma aislada, midiendo su desempeño (histogramas, distancia de Hamming, diagramas de dispersión) sin que el transporte introduzca variables adicionales.

8 Resumen del procedimiento

1. El publisher cifra la imagen mediante difusión (mapa logístico) y confusión (sistema de Rössler).
2. Serializa el resultado y los parámetros necesarios a JSON.
3. Abre una conexión TCP al broker en el puerto 8883.
4. Ejecuta el *handshake* TLS: valida el certificado del broker contra `ca.crt` y deriva las claves de sesión K_c y K_s .
5. Envía el paquete MQTT CONNECT ya cifrado y recibe CONNACK.
6. Publica en `chaoskeystream/keys` y `chaoskeystream/data` con QoS 1 y `retain` activo.
7. El broker descifra, almacena y reenvía cada mensaje al subscriber a través de la sesión TLS de este.
8. El subscriber deserializa el JSON y reconstruye la imagen mediante sincronización con el sistema maestro.
9. El publisher cierra la sesión; los mensajes retenidos permanecen disponibles en el broker.